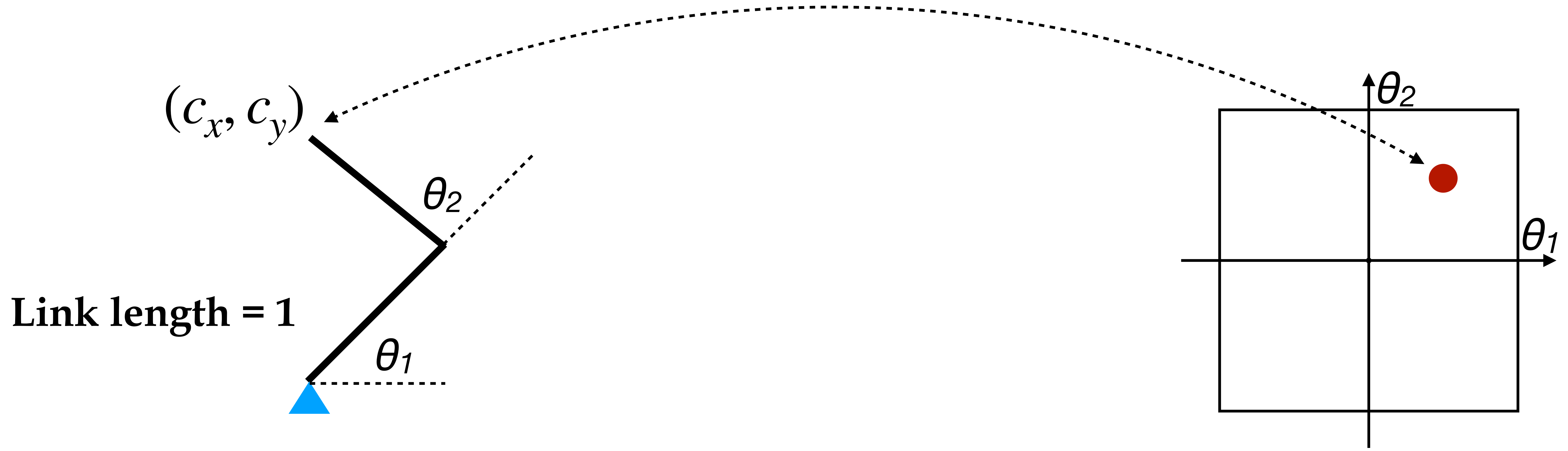
Lecture 3. Kinematic and Dynamic Models

Kinematic Manipulator Models

- Cartesian space vs configuration space
- Cartesian space model
- Joint space model
- Cartesian-Joint model

Cartesian Space vs Configuration Space

Cartesian Space **Configuration Space**

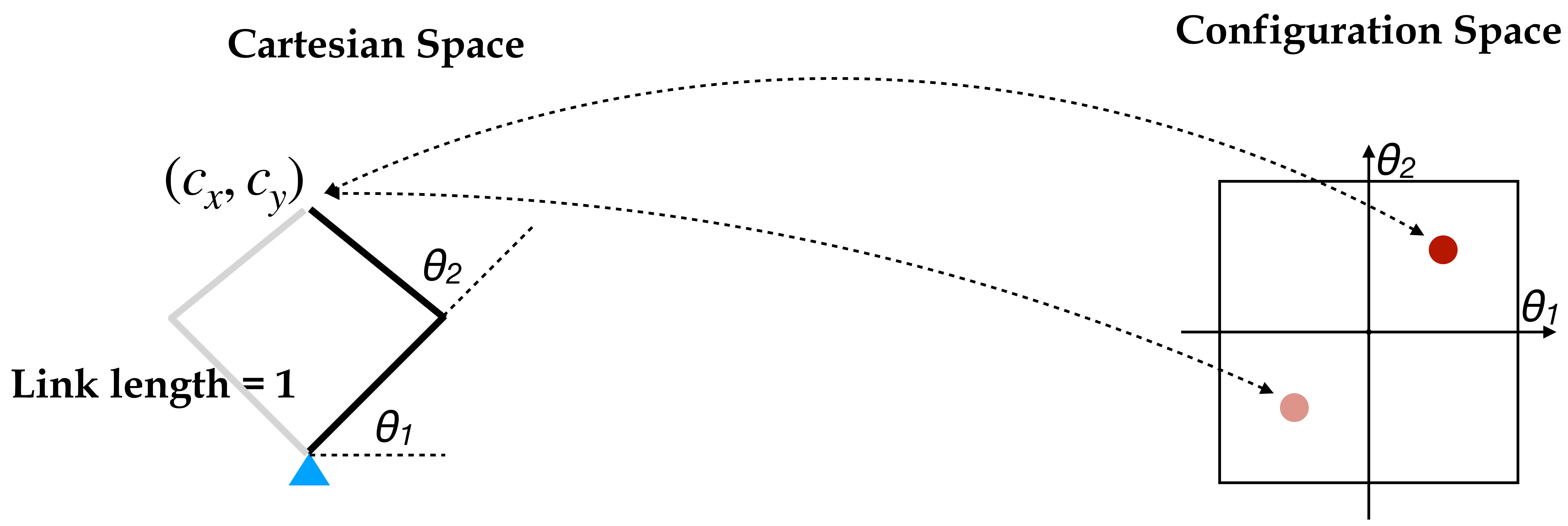


Forward Kinematics

$$c_x = \cos\theta_1 + \cos(\theta_1 + \theta_2)$$

$$c_y = \sin\theta_1 + \sin(\theta_1 + \theta_2)$$

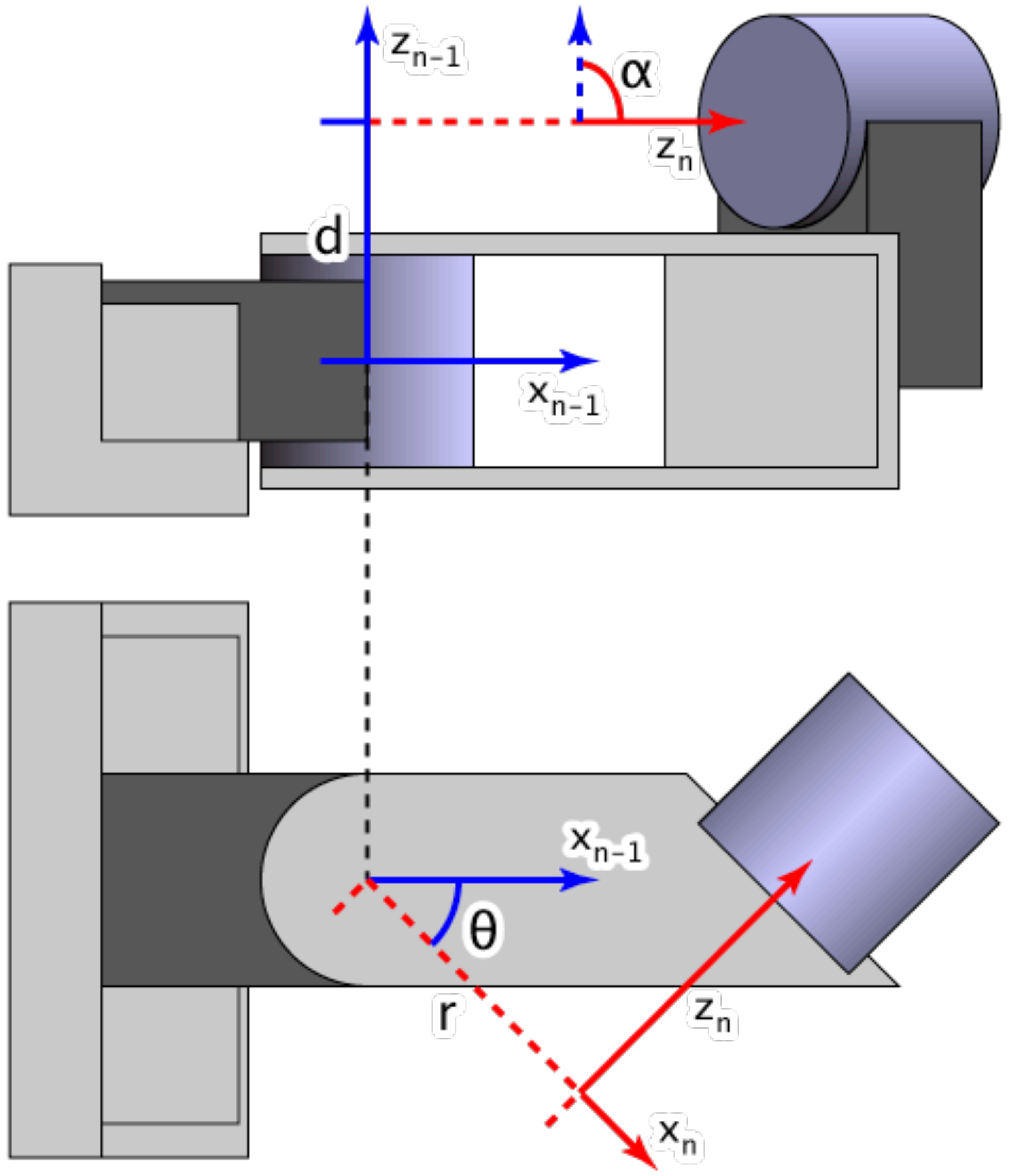
Cartesian Space vs Configuration Space



Inverse Kinematics are not unique!

D-H Parameters

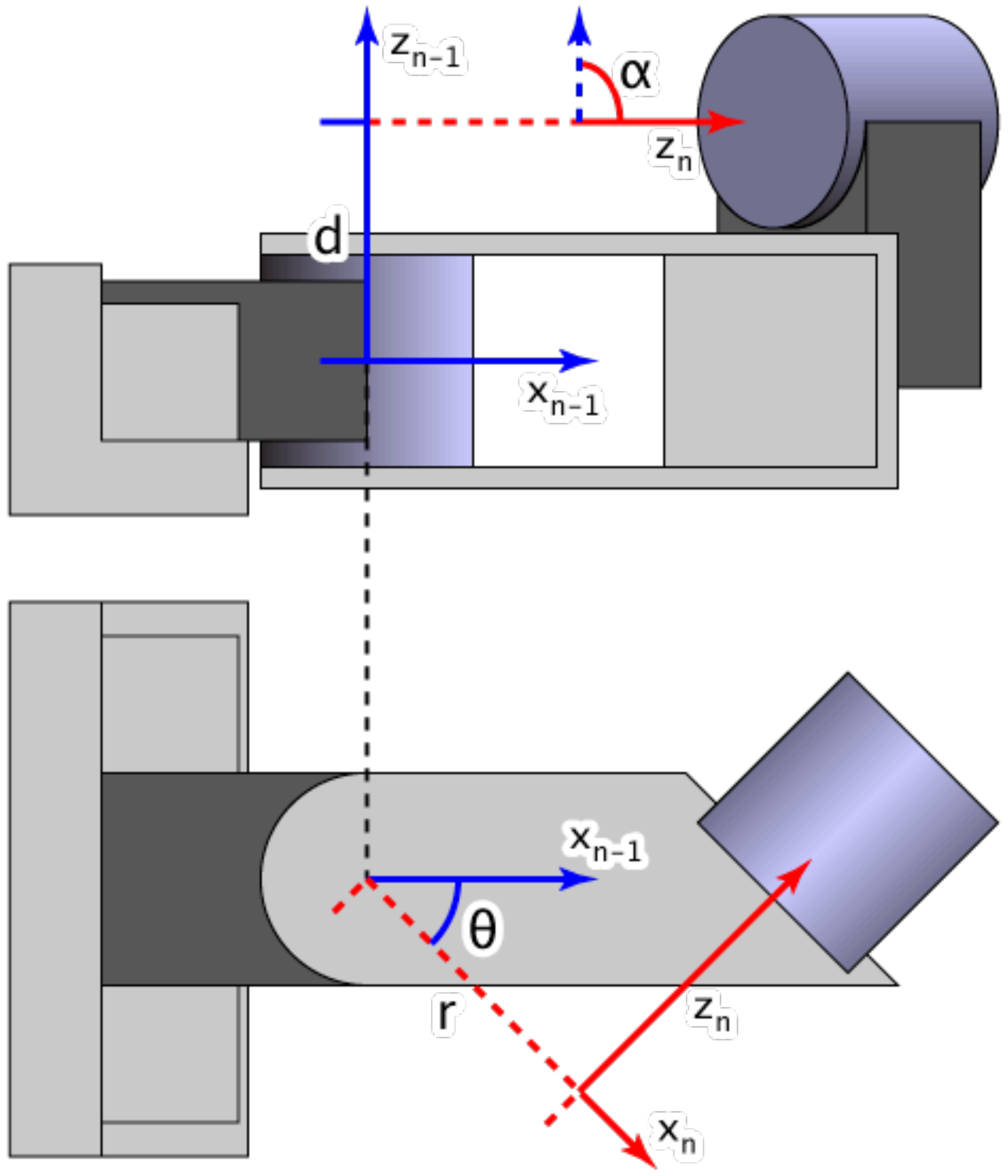
- A general way to encode the mapping from the configuration space to the Cartesian space: **D-H Parameters**
- For each joint, we assign a reference frame:
  - z-axis: in the direction of the joint axis
  - x-axis: parallel to the common normal $x_n = z_n \times z_{n-1}$
  - y-axis: follow from right handed coordinate system



https://en.wikipedia.org/wiki/Denavit–Hartenberg_parameters

D-H Parameters

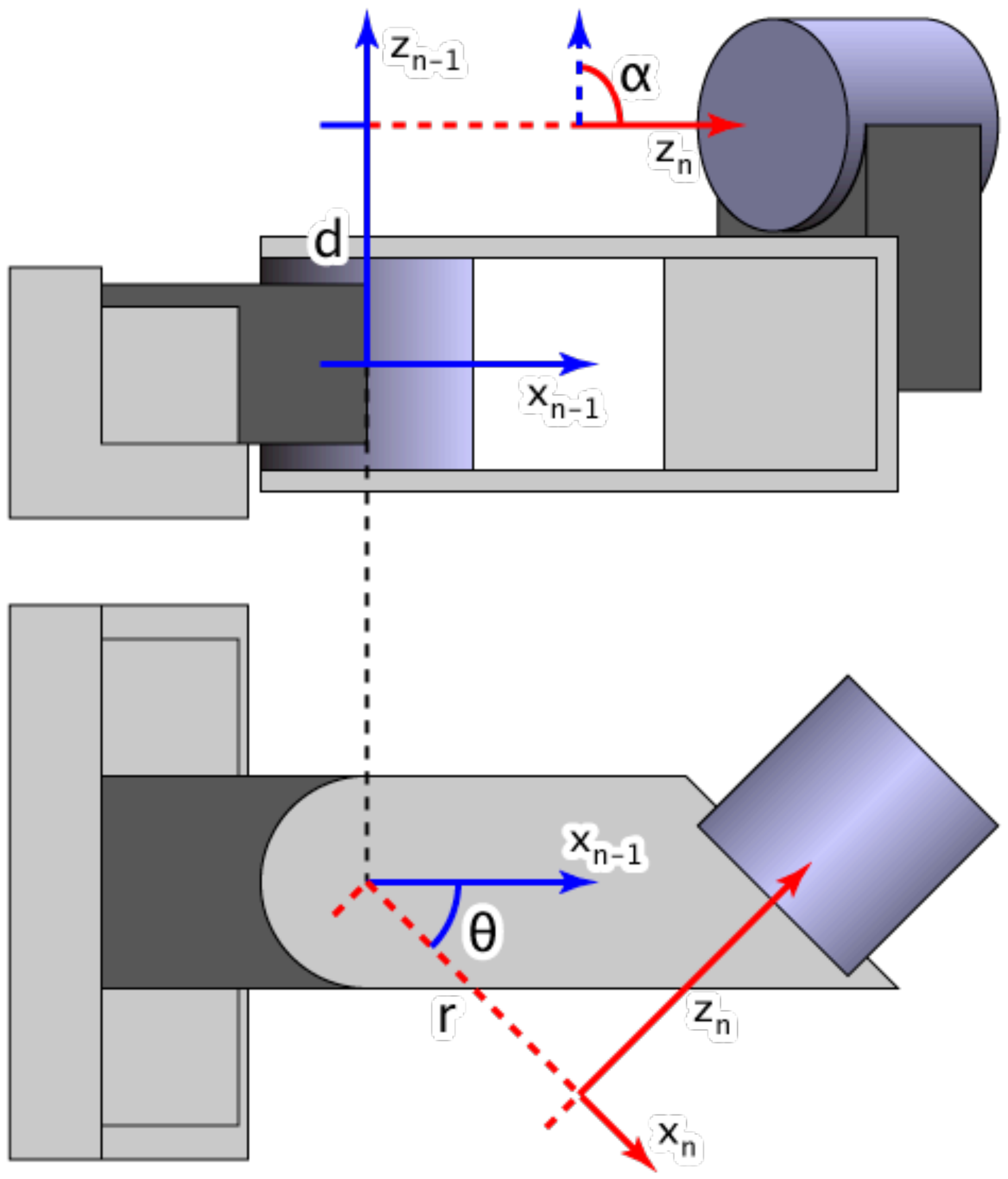
- Transformation parameters from one joint to another:
  - d: offset along previous z-axis to the common normal
  - θ: angle about previous z-axis from old x to new x
  - r: length of common normal
  - α: angle about common normal from old z-axis to new z-axis



https://en.wikipedia.org/wiki/Denavit–Hartenberg_parameters

D-H Parameters

$$
{}^{n-1}T_n = \left[\begin{array}{ccc|c} \cos\theta_n & -\sin\theta_n\cos\alpha_n & \sin\theta_n\sin\alpha_n & r_n\cos\theta_n \\ \sin\theta_n & \cos\theta_n\cos\alpha_n & -\cos\theta_n\sin\alpha_n & r_n\sin\theta_n \\ 0 & \sin\alpha_n & \cos\alpha_n & d_n \\ \hline 0 & 0 & 0 & 1 \end{array}\right] = \left[\begin{array}{ccc|c} & R & & T \\ \hline 0 & 0 & 0 & 1 \end{array}\right]
$$

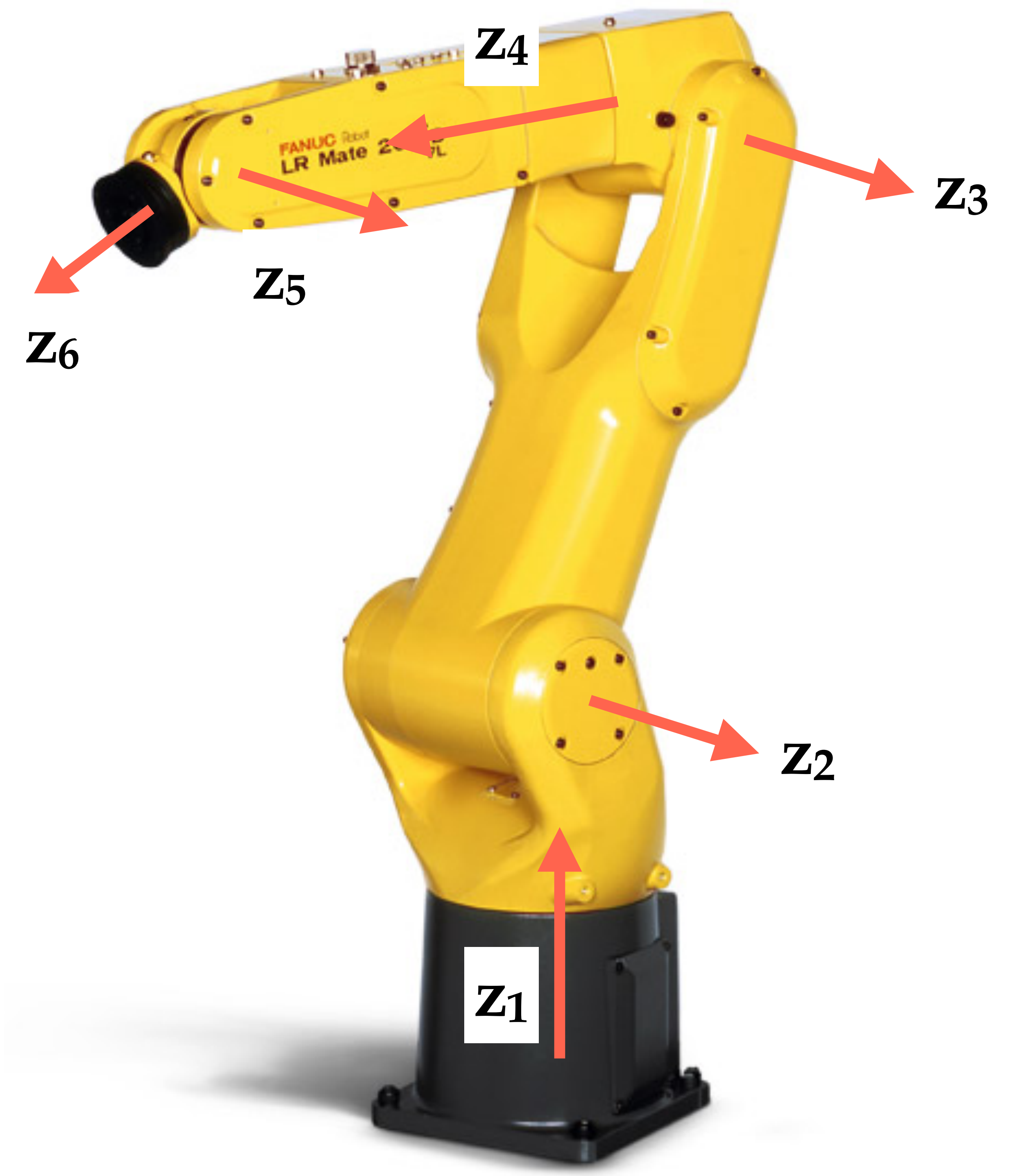


https://en.wikipedia.org/wiki/Denavit–Hartenberg_parameters

D-H Parameters

- Multi-link Transformation
  - Transformation from link i to link j

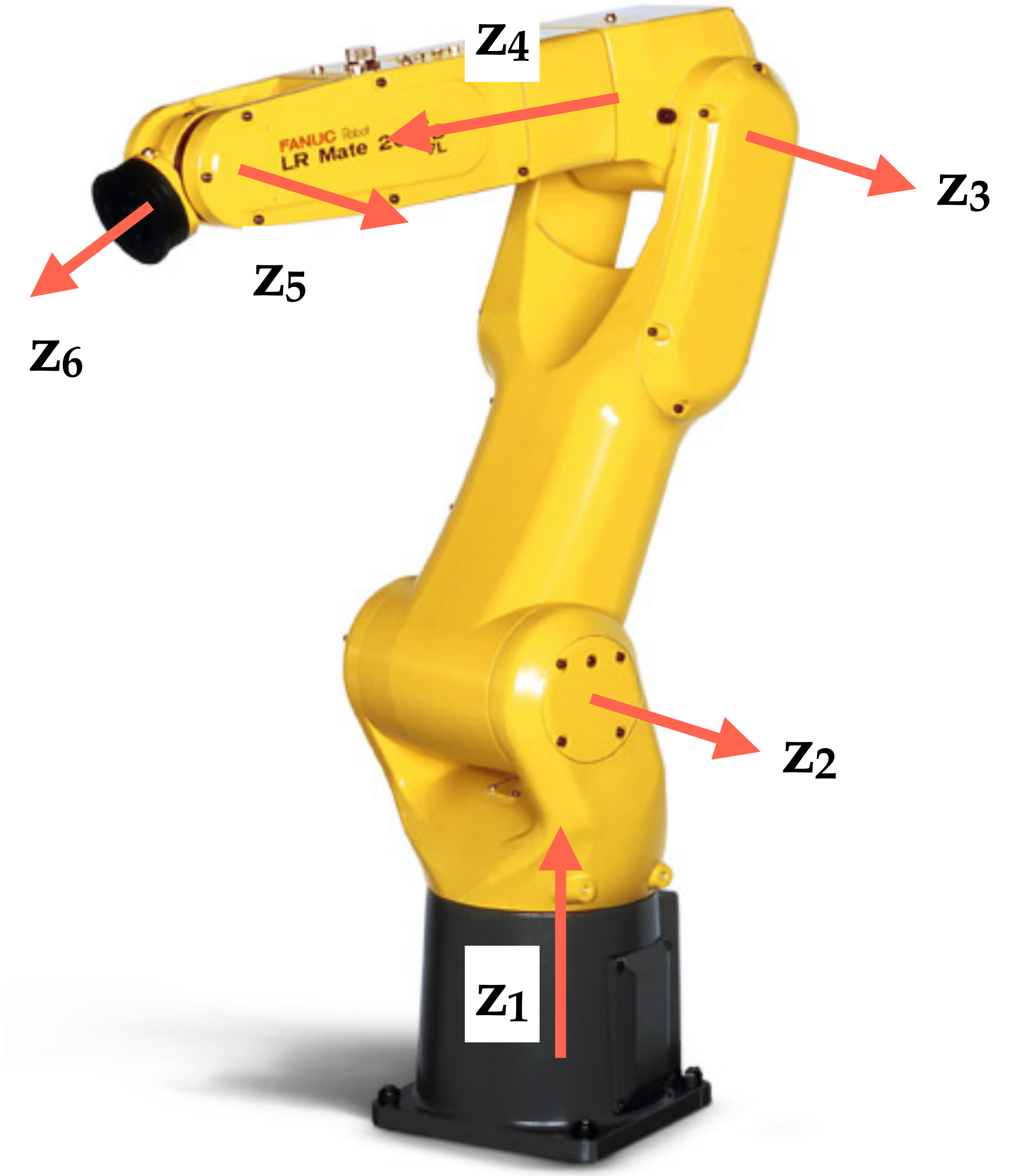
$$M_{i,j} = \Pi_{k=i+1}^{j} \; {}^{k-1}T_k \qquad = \left[\begin{array}{ccc|c} & R & & T \\ \hline 0 & 0 & 0 & 1 \end{array}\right]$$



Example: 6 DOF Industrial Robot Arm

These are configuration space variables

θ	d	r	α
0	0	0.05	- π
- π	0	0.44	2π
0	0	0.035	- π
0	-0.42	0	π
0	0	0	- π
0	-0.08	0	2π



Example: 6 DOF Industrial Robot Arm

https://github.com/changliuliu/RobotKinematics

This script is to illustrate robot kinematics.

Step 1: Choose a robot

```
robot = robotproperty( M16iB ▾ )
theta = robot.DH(:,1);
```

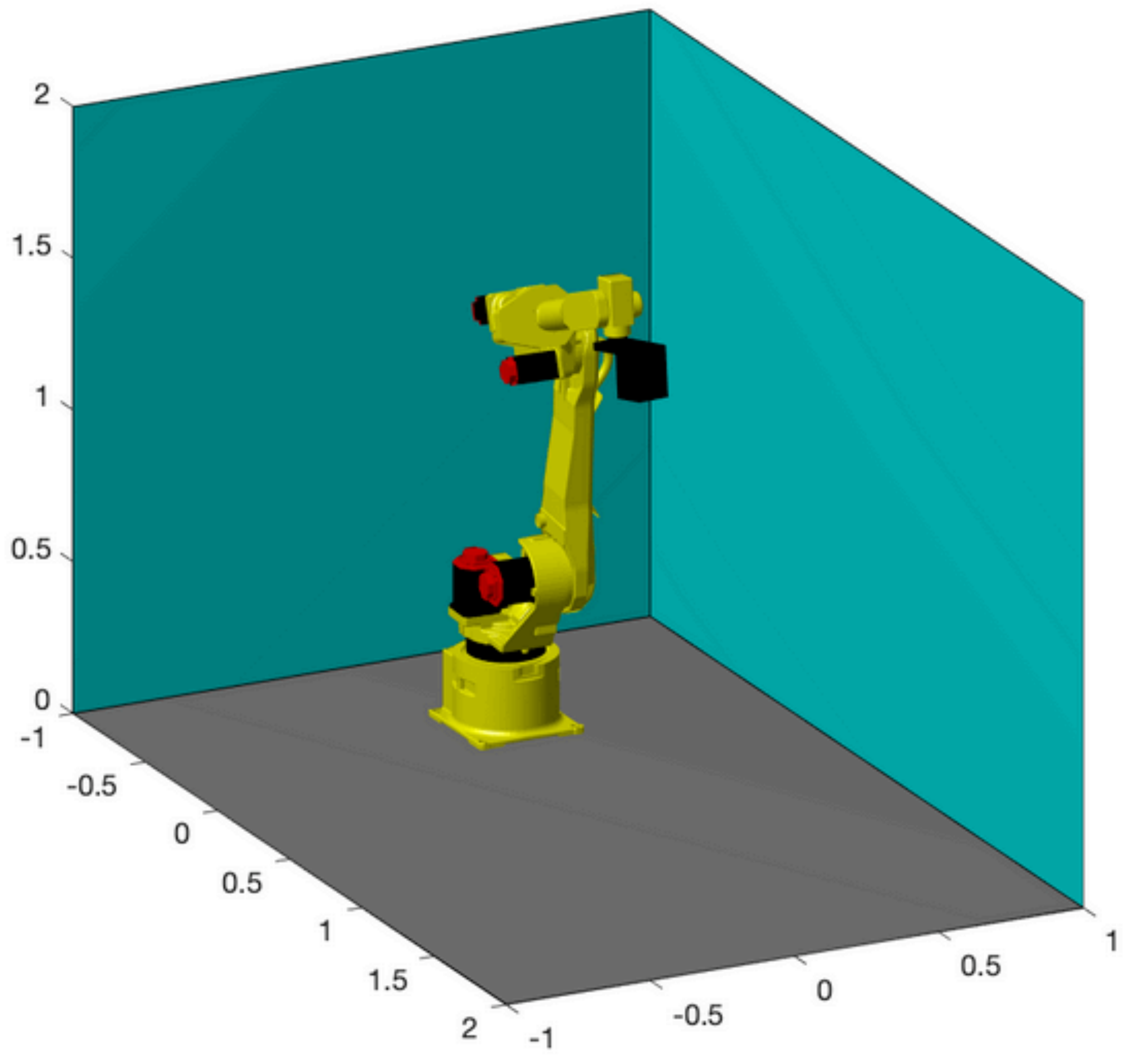
Step 2: Choose joint angles (offset in rad)

```
robot.DH(:, 1) = theta + [
      -0.05  ;
      -0.06  ;
       0.43  ;
      -0.02  ;
      -0.21  ;
      -0.28  ;
    ];
```

Step 3: Now let's draw the robot!

```
Draw(robot, CAD ▾ );
```

```
robot = struct with fields:
      name: "M16iB"
     nlink: 6
        DH: [6×4 double]
      base: [3×1 double]
```



Jacobian

- How does the change of the configuration space affect the change of the Cartesian space?

$$c = FK(\theta)$$

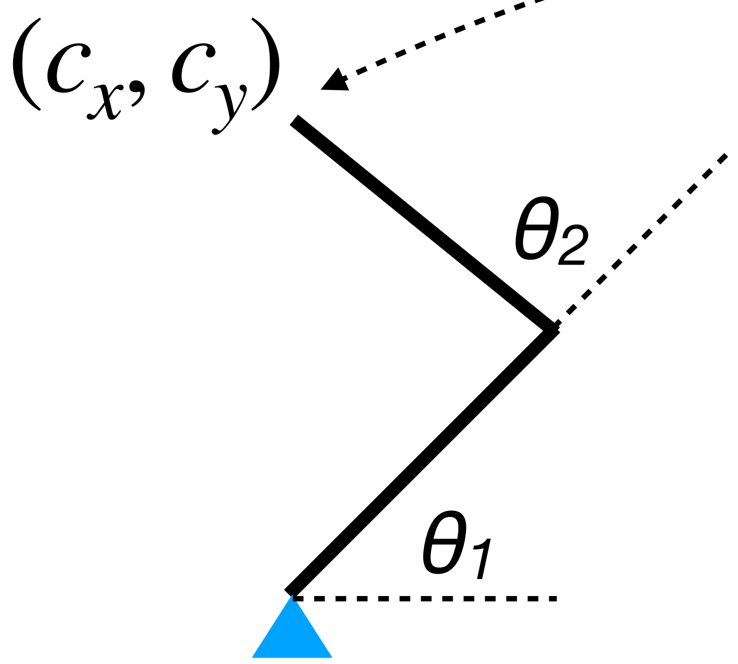
Translational and Angular velocity ⟶ $$\dot{c} = J(\theta)\dot{\theta}$$

$$\ddot{c} = J(\theta)\ddot{\theta} + \underbrace{\nabla_\theta J(\theta)(\dot{\theta})^2}_{H(\theta,\dot{\theta})}$$

Jacobian

Cartesian Space

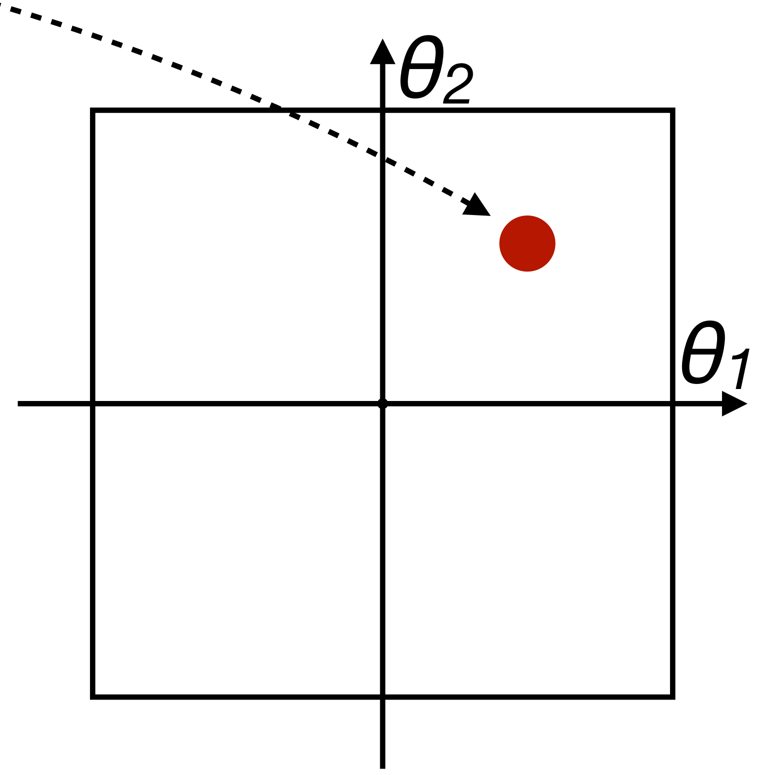
Configuration Space



Forward Kinematics

$$c_x = \cos\theta_1 + \cos(\theta_1 + \theta_2)$$

$$c_y = \sin\theta_1 + \sin(\theta_1 + \theta_2)$$



$$\dot{c}_x = [-\sin\theta_1 - \sin(\theta_1 + \theta_2)]\dot{\theta}_1 + [-\sin(\theta_1 + \theta_2)]\dot{\theta}_2$$

$$\dot{c}_y = [\cos\theta_1 + \cos(\theta_1 + \theta_2)]\dot{\theta}_1 + [\cos(\theta_1 + \theta_2)]\dot{\theta}_2$$

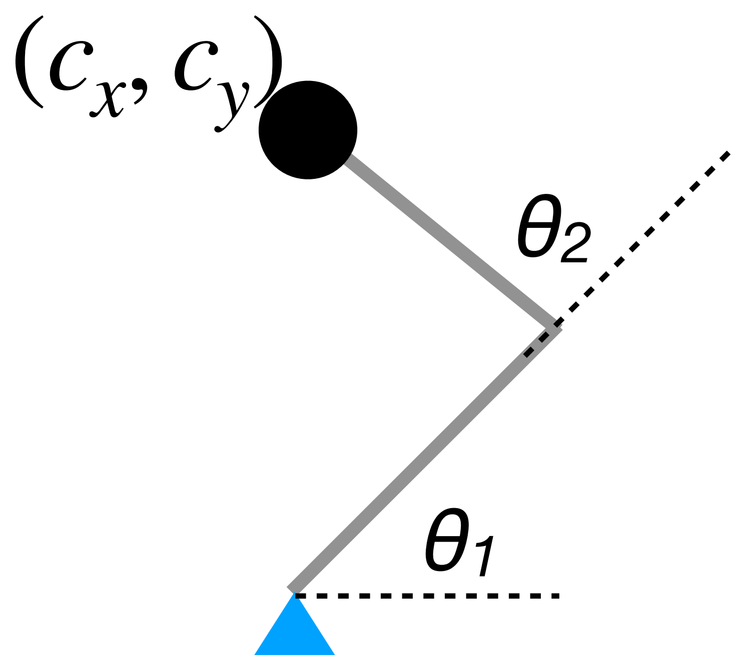
$$\dot{c}_w = \dot{\theta}_1 + \dot{\theta}_2$$

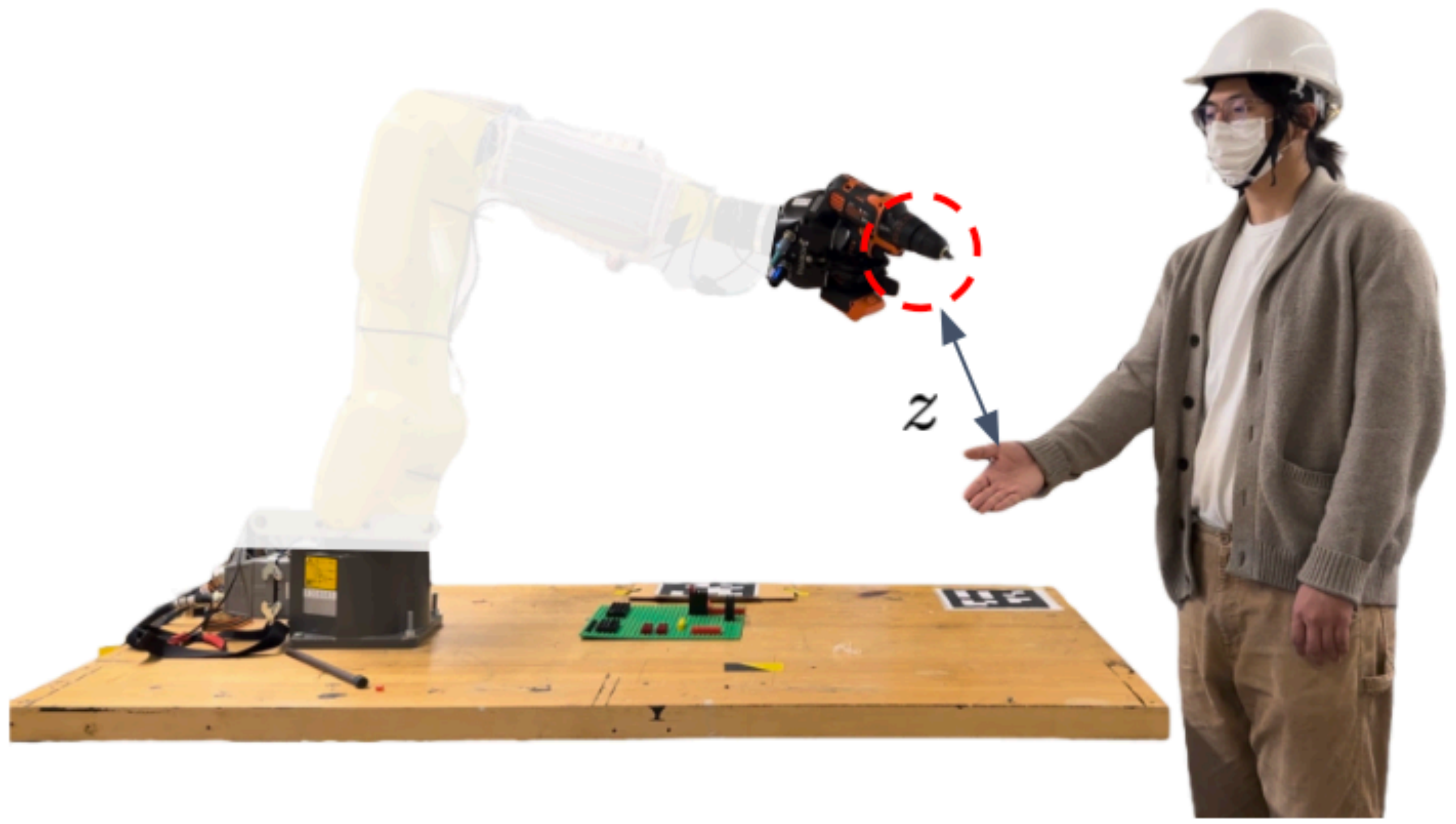
Remark

- Different robot structure may imply different set of parameters in the configuration space
- Generally speaking
  - For rotational joints, the angle θ is in the configuration space
  - For translational joints, the distance d or r are in the configuration space
- In the following discussion, we call configuration space as joint space and only consider rotational joints.

Cartesian Space Model

- Position only single integrator model (dimension $n = 2{,}3$)
  - State $x \in \mathbb{R}^n$: end-effector position
  - Control $u \in \mathbb{R}^n$: end-effector velocity
  - Dynamic equation
    - $\dot{x} = u$
    - $x_{k+1} = x_k + \Delta t \; u_k$

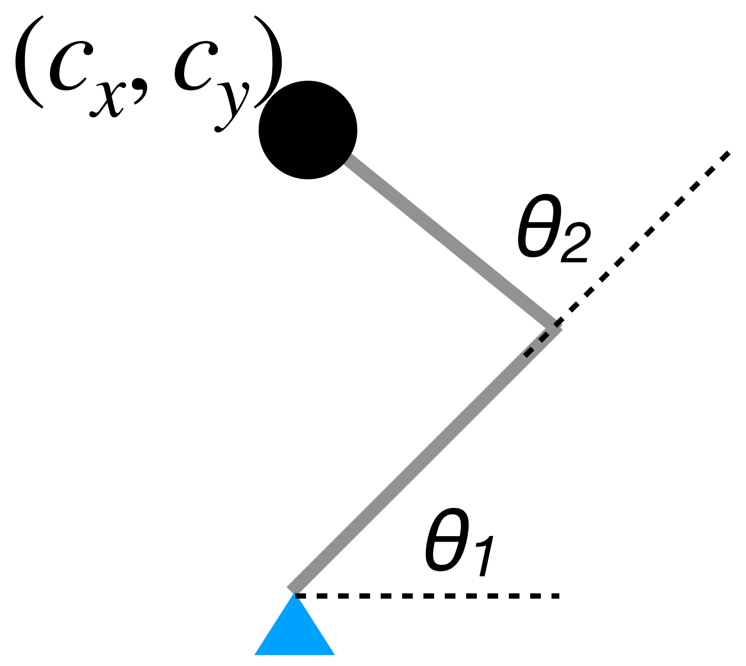


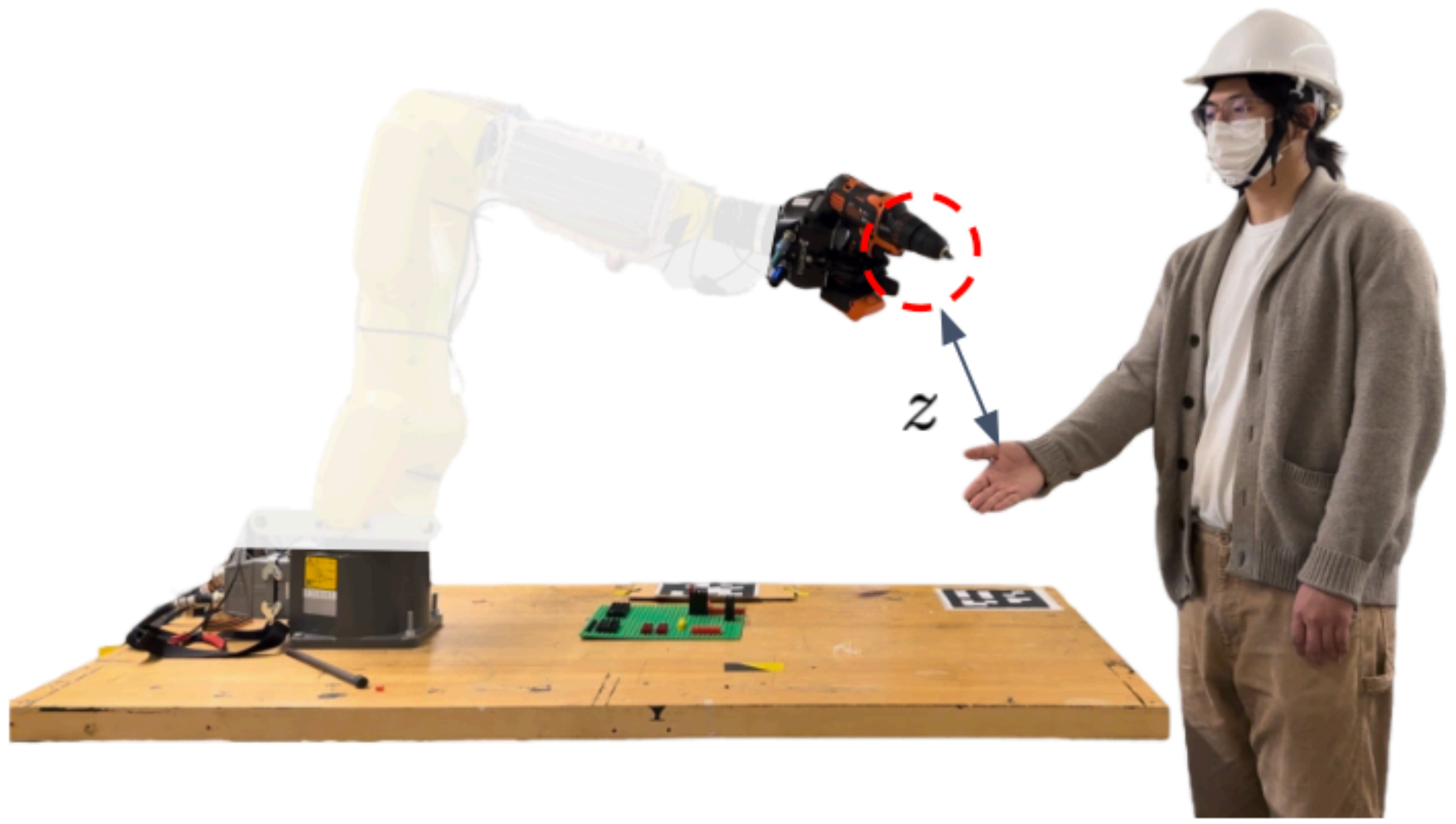


Note the similarity with single integrator vehicle model!

Cartesian Space Model

- Position only double integrator model (dimension $n = 2{,}3$)
  - State $x \in \mathbb{R}^{2n}$: end-effector position and velocity
  - Control $u \in \mathbb{R}^{n}$: end-effector acceleration
  - Dynamic equation
    - $\dot{x} = \begin{bmatrix} 0_n & I_n \\ 0_n & 0_n \end{bmatrix} x + \begin{bmatrix} 0_n \\ I_n \end{bmatrix} u$
    - $x_{k+1} = \begin{bmatrix} I_n & \Delta t I_n \\ 0_n & I_n \end{bmatrix} x_k + \begin{bmatrix} 0_n \\ \Delta t I_n \end{bmatrix} u_k$

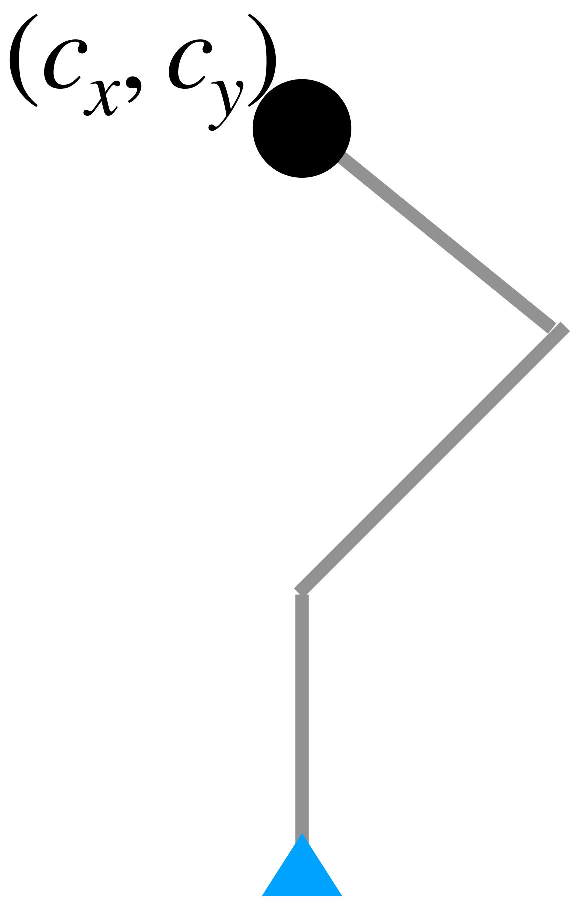


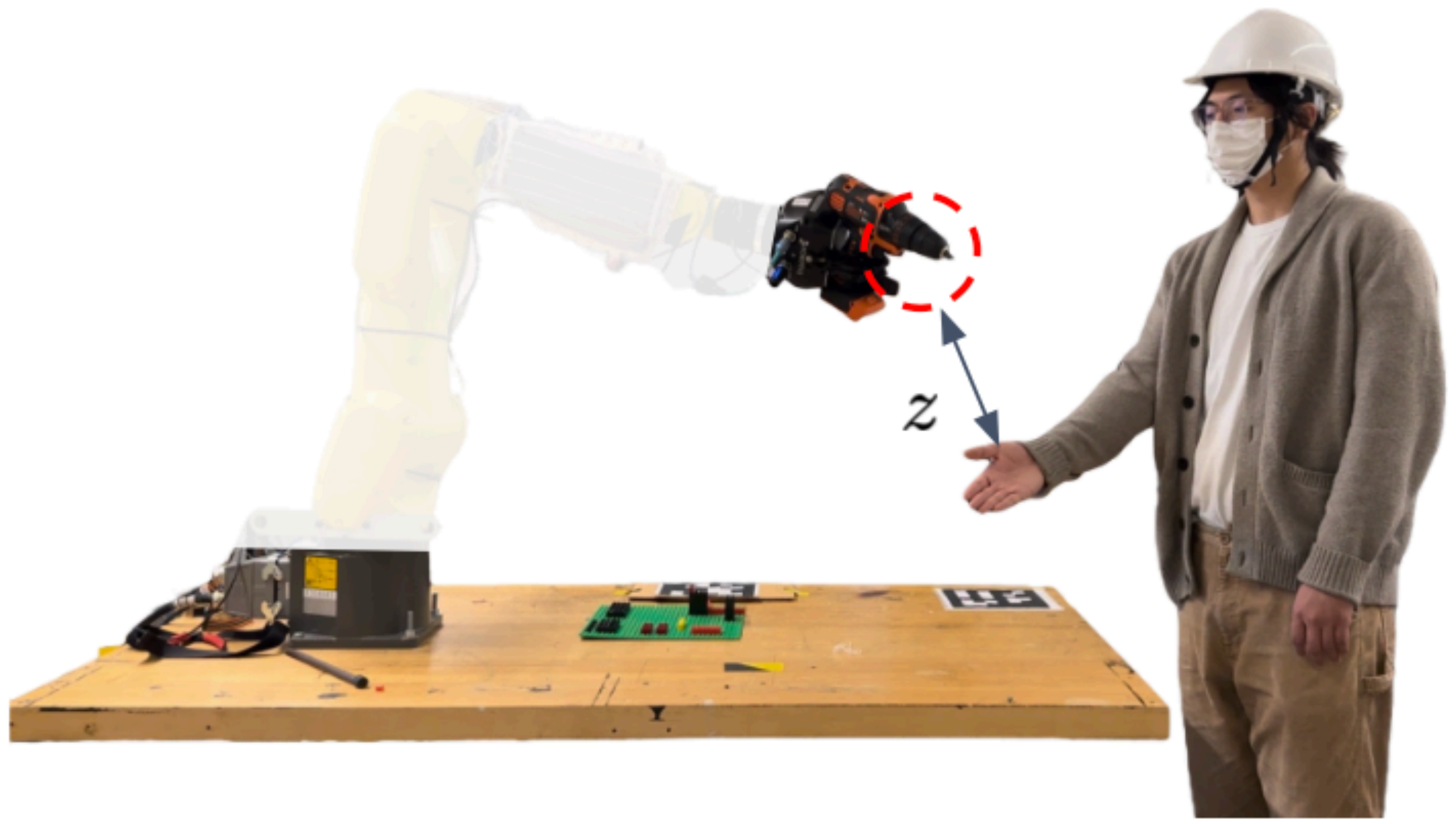


Note the similarity with double integrator vehicle model!

Cartesian Space Model

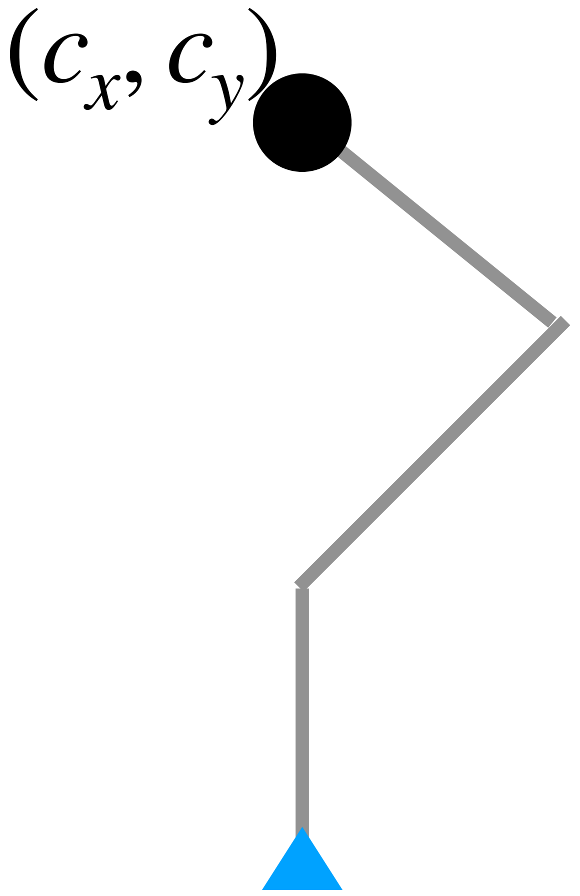
- Pose single integrator model (dimension $n = 2,3$)
  - State $x \in \mathbb{R}^{3n-3}$: end-effector position and angle
  - Control $u \in \mathbb{R}^{3n-3}$: end-effector velocity and turning rate
  - Dynamic equation
    - $\dot{x} = u$
    - $x_{k+1} = x_k + \Delta t\ u_k$

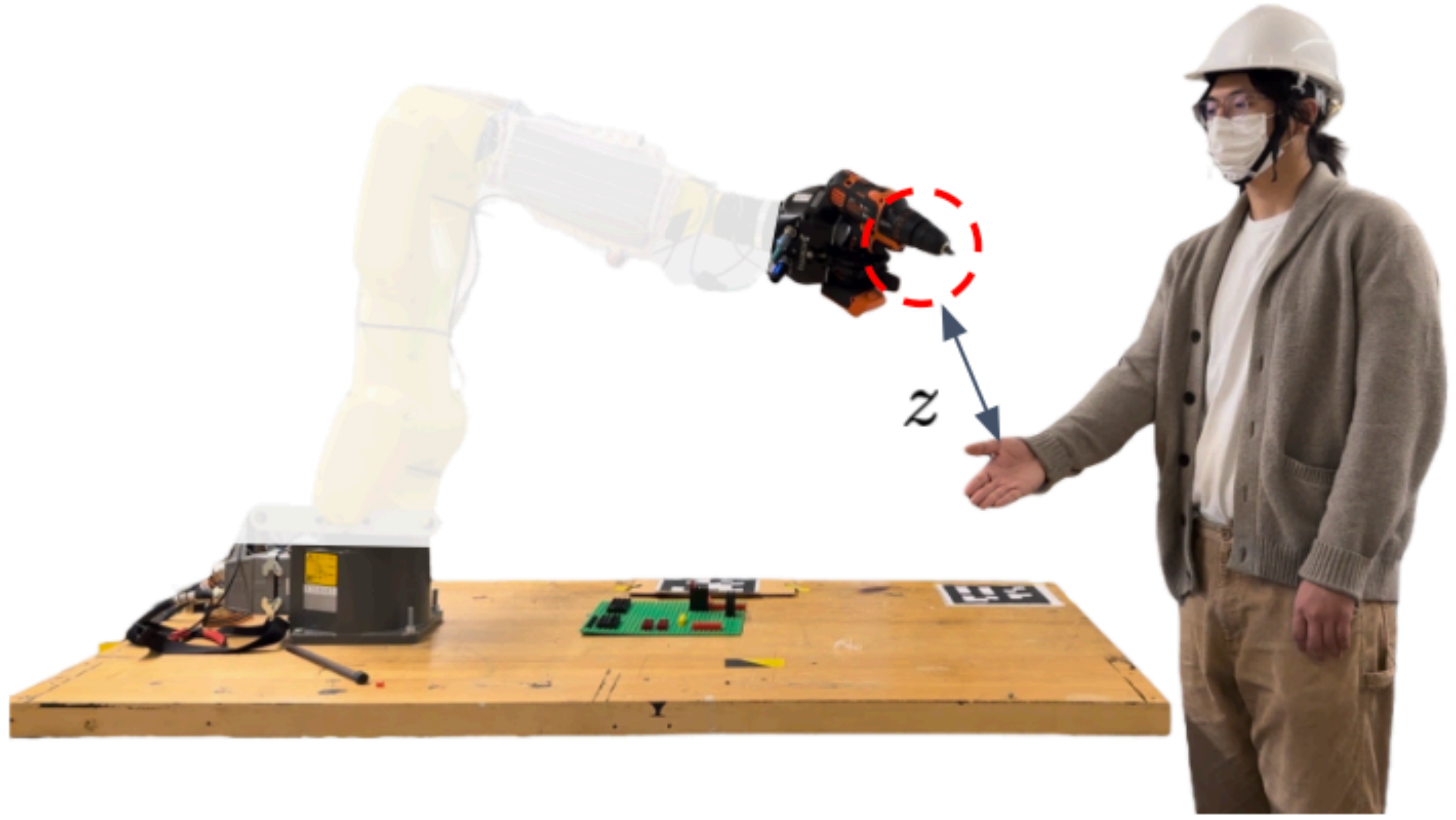




Cartesian Space Model

- Pose double integrator model (dimension $n = 2{,}3$)
  - State $x \in \mathbb{R}^{6n-6}$: end-effector position and angle and their velocities
  - Control $u \in \mathbb{R}^{3n-3}$: end-effector acceleration and angular acceleration
  - Dynamic equation
    - $\dot{x} = \begin{bmatrix} 0_{3n-3} & I_{3n-3} \\ 0_{3n-3} & 0_{3n-3} \end{bmatrix} x + \begin{bmatrix} 0_{3n-3} \\ I_{3n-3} \end{bmatrix} u$
    - $x_{k+1} = \begin{bmatrix} I_{3n-3} & \Delta t I_{3n-3} \\ 0_{3n-3} & I_{3n-3} \end{bmatrix} x_k + \begin{bmatrix} 0_{3n-3} \\ \Delta t I_{3n-3} \end{bmatrix} u_k$





Cartesian Space Model

Advantage

Simple, linear

Disadvantage

The controls are “imaginary control inputs”. We may not always be able to find an “actual control signal” to track them.

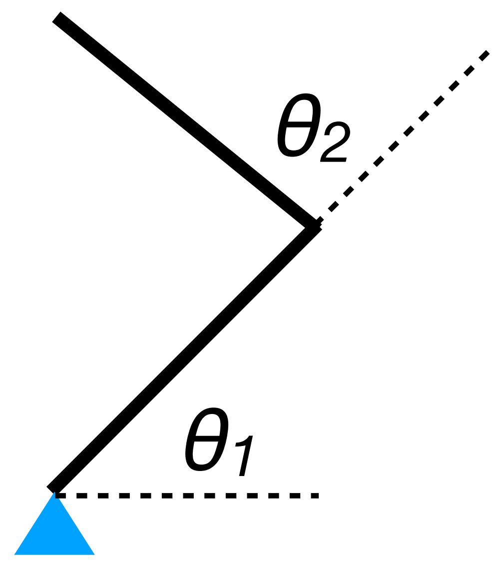
Joint Space Model

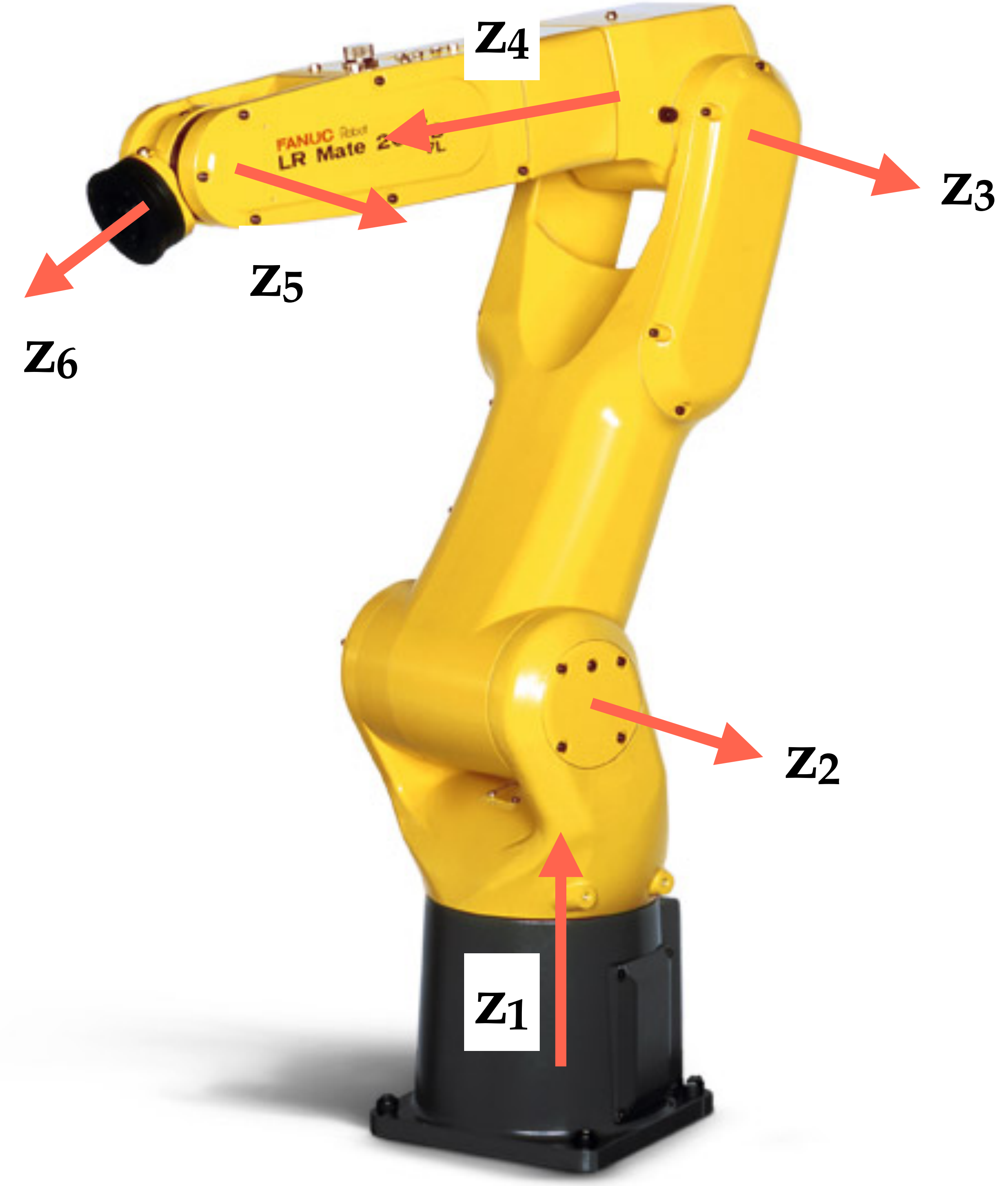
- Single Integrator

$$x = \theta, u = \dot{\theta} \Rightarrow \dot{x} = u$$

- Double Integrator

$$x = [\theta, \dot{\theta}], u = \ddot{\theta} \Rightarrow \dot{x} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ I \end{bmatrix} u$$





Joint Space Model

Advantage	Disadvantage
Simple, linear Minimum control mismatch	System objectives are usually specified in the Cartesian space

Cartesian-Joint Model

- First-order model
  - State $x \in \mathbb{R}^{3n-3}$: Cartesian pose
  - Control $u \in \mathbb{R}^m$: Joint velocity
  - Dynamic equation
    - $\dot{x} = J(\theta)u$, where $\theta = FK^{-1}(x)$
    - $x_{k+1} = x_k + \Delta t \; J(\theta)u_k$

Cartesian-Joint Model

- Second-order model
  - State $x \in \mathbb{R}^{6n-6}$: Cartesian pose c and velocity $\dot{c}$
  - Control $u \in \mathbb{R}^{m}$: Joint acceleration
  - Dynamic equation
    - $\dot{x} = \begin{bmatrix} 0_{3n-3} & I_{3n-3} \\ 0_{3n-3} & 0_{3n-3} \end{bmatrix} x + \begin{bmatrix} 0_{3n-3} \\ I_{3n-3} \end{bmatrix} (J(\theta)u + H(\theta, \dot{\theta}))$ **Note** $\theta = FK^{-1}(c)$, $\dot{\theta} = J(\theta)^{-1}\dot{c}$
    - $x_{k+1} = \begin{bmatrix} I_{3n-3} & \Delta t I_{3n-3} \\ 0_{3n-3} & I_{3n-3} \end{bmatrix} x + \begin{bmatrix} 0_{3n-3} \\ \Delta t I_{3n-3} \end{bmatrix} (J(\theta)u_k + H(\theta, \dot{\theta}))$

Cartesian-Joint Model

Advantage

Can easily address various objectives in the Cartesian space

Disadvantage

The dynamic model is nonlinear

Kinematic Manipulator Models

Model	State	Control	Continuous Time Dynamics	Discrete Time Dynamics
C-Single Integrator				
C-Double Integrator				
J-Single Integrator				
J-Double Integrator				
C-J First Order				
C-J Second Order				

Dynamic Models

- Equations of Motion
- Joint space dynamic model

Equations of Motion

$$\tau = H(\theta)\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + \tau_g(\theta) + J(\theta)^T f_{ext}$$

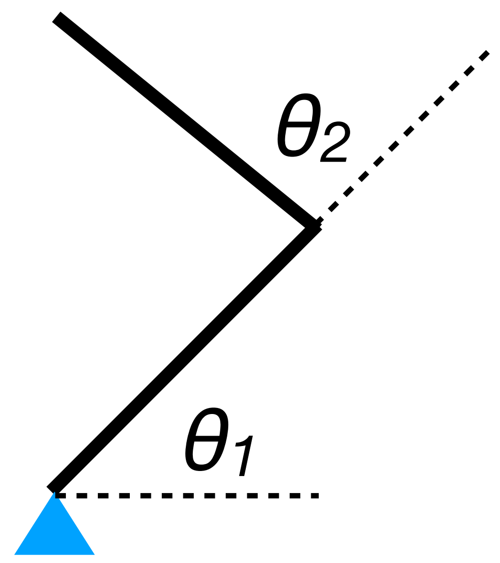
- $H(\theta)$: joint space inertia matrix (symmetric and positive definite)
- $C(\theta, \dot{\theta})\dot{\theta}$: Coriolis and centrifugal forces
- $\tau_g(\theta)$: gravity
- f_{ext}: external force applied to the end-effector

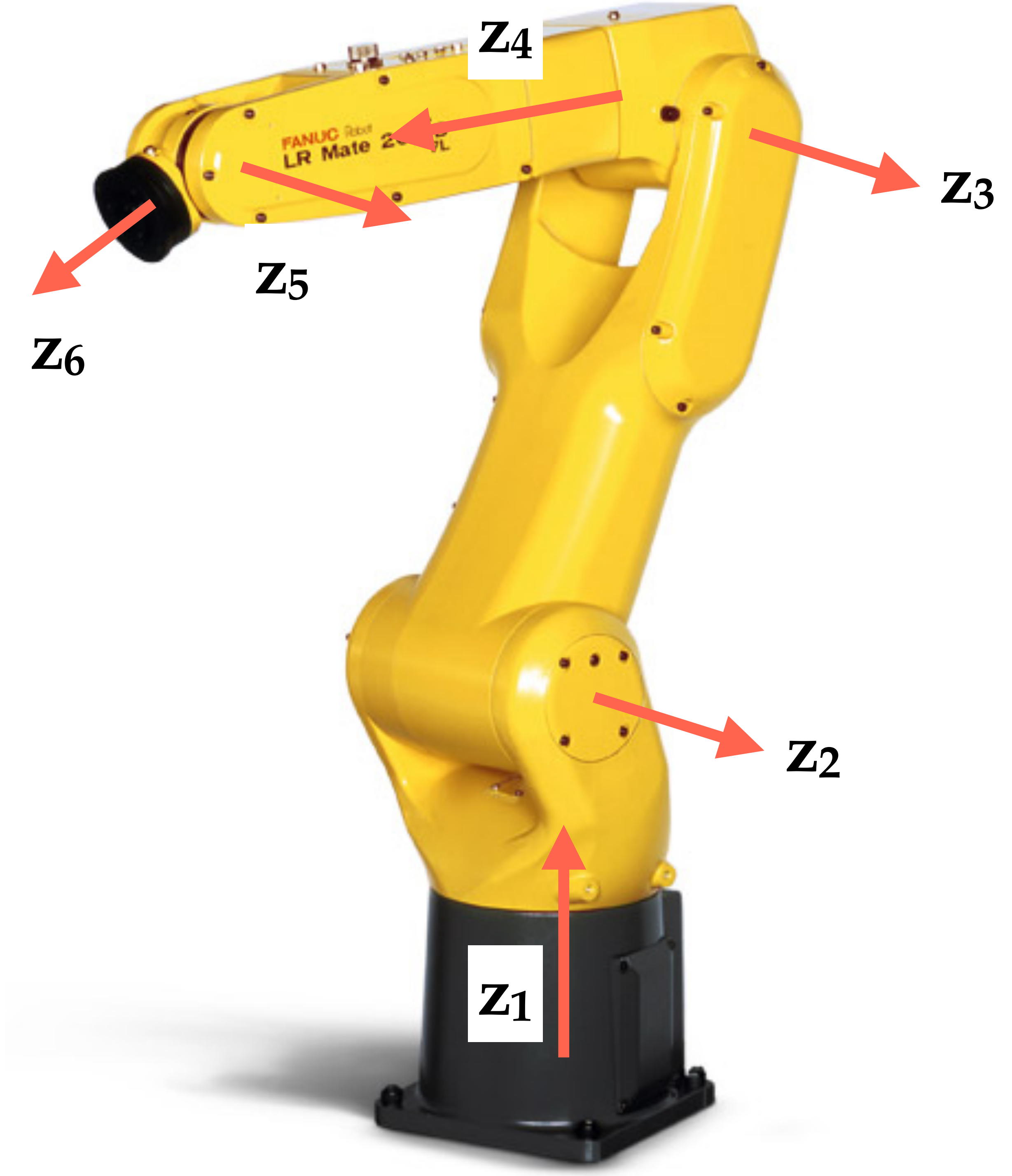
Joint Space Dynamic Model

- Recall that the kinematic model is

$$x = [\theta, \dot{\theta}], u = \ddot{\theta} \Rightarrow \dot{x} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ I \end{bmatrix} u$$

- Dynamic model:
  - State $x = [\theta; \dot{\theta}]$
  - Control $u = \tau$
  - $\dot{x} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ H(\theta)^{-1} \end{bmatrix} (u - C(\theta, \dot{\theta})\dot{\theta} - \tau_g(\theta) - J(\theta)^T f_{ext})$



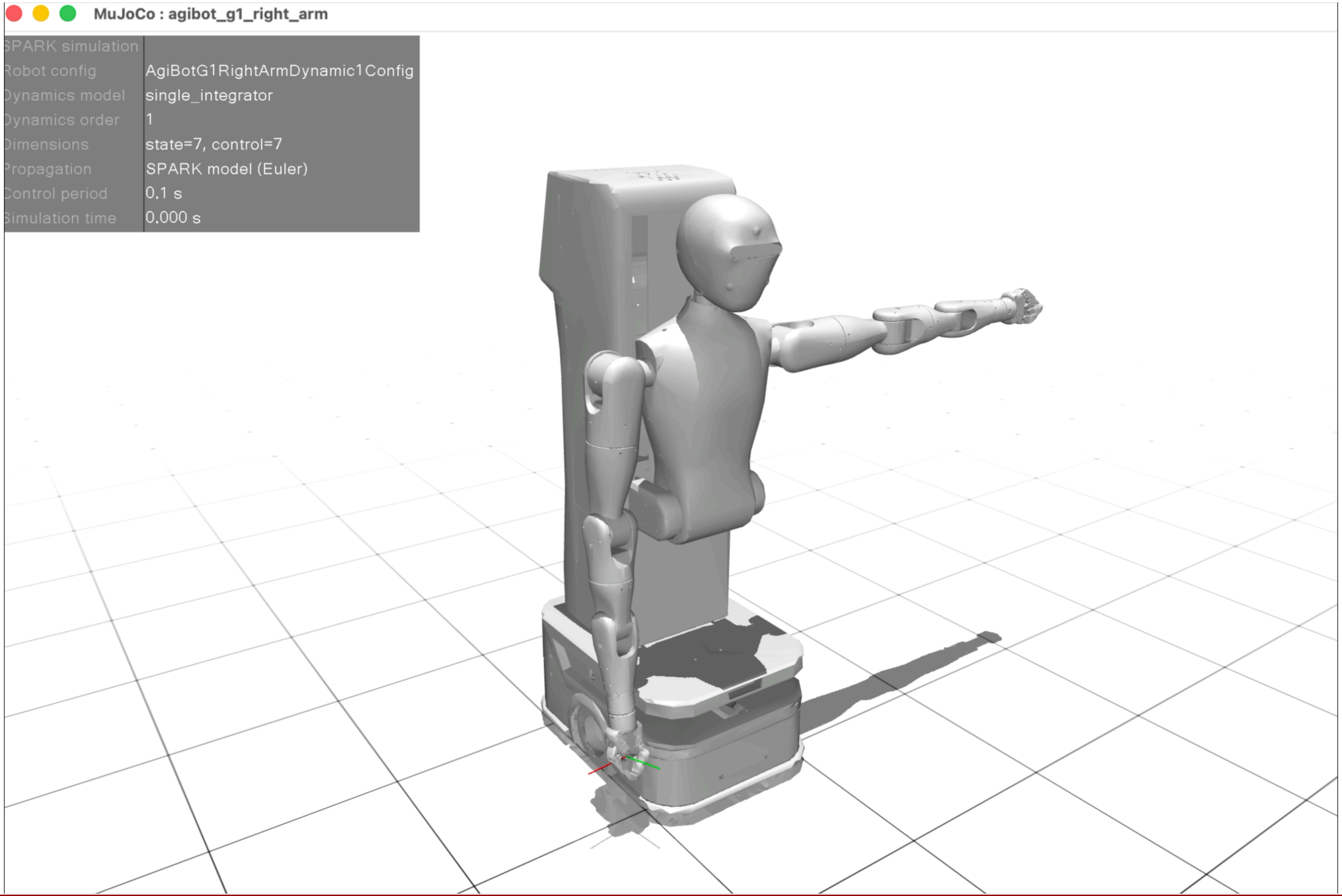


$$\tau = H(\theta)\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + \tau_g(\theta) + J(\theta)^T f_{ext}$$

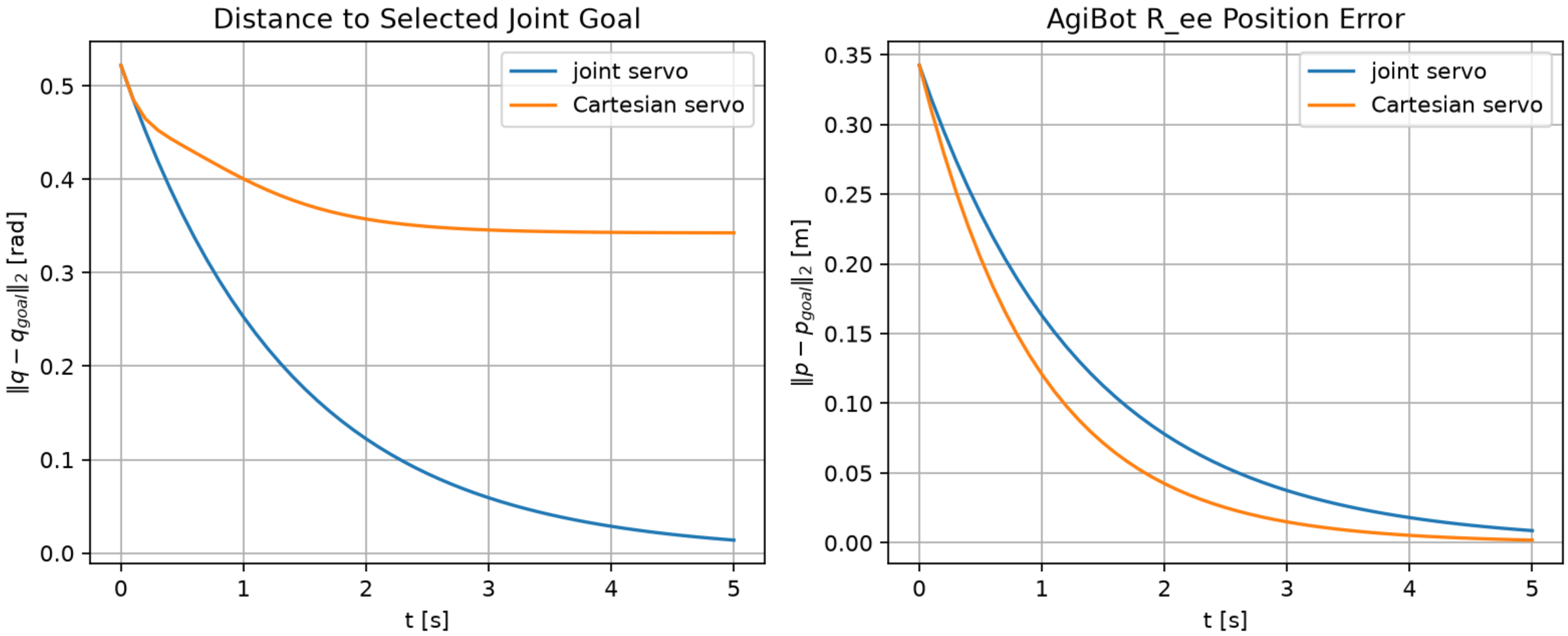
Remark

- Although the joint acceleration and the torque is a bijection, dynamic modeling is necessary in certain scenarios:
  - To correctly characterize the limits in torque
  - To correctly account for contacts through f_{ext}

Simulation

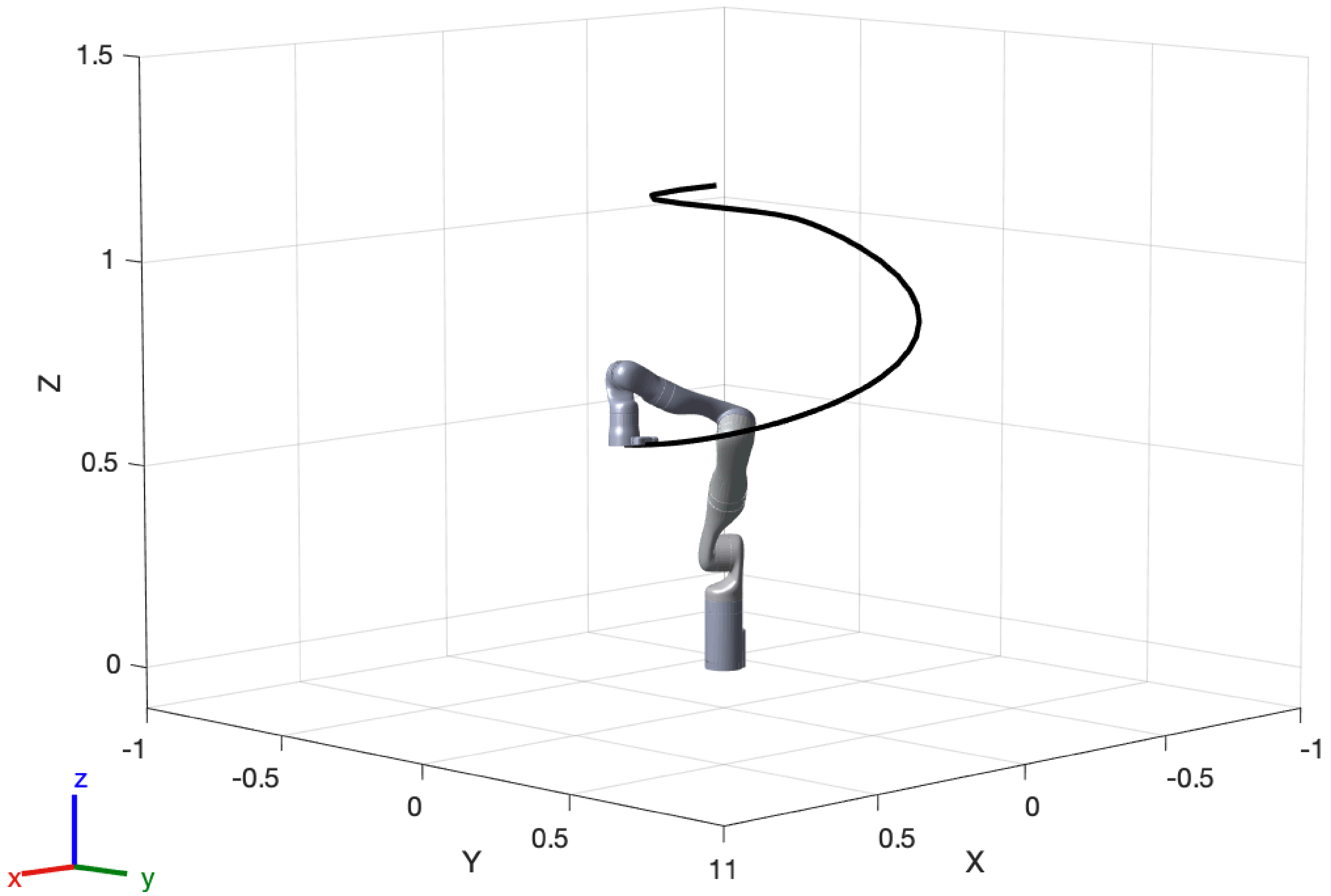

MuJoCo : agibot_g1_right_arm
SPARK simulation
Robot config AgiBotG1RightArmDynamic1Config
Dynamics model single_integrator
Dynamics order 1
Dimensions state=7, control=7
Propagation SPARK model (Euler)
Control period 0.1 s
Simulation time 0.000 s

Comparison

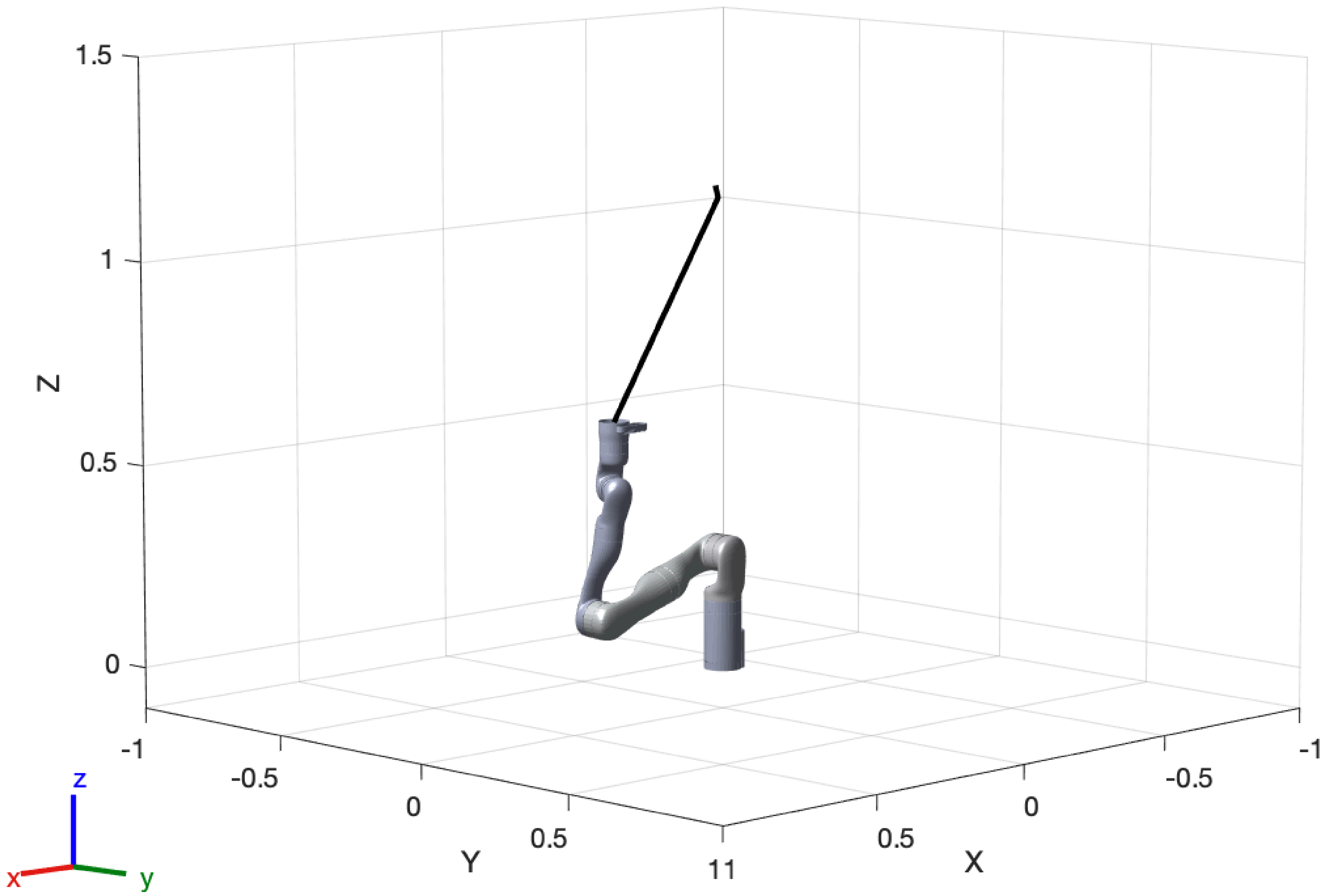

Distance to Selected Joint Goal
joint servo
Cartesian servo
$\|q - q_{goal}\|_2$ [rad]
0.0
0.1
0.2
0.3
0.4
0.5
0
1
2
3
4
5
t [s]
AgiBot R_ee Position Error
joint servo
Cartesian servo
$\|p - p_{goal}\|_2$ [m]
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0
1
2
3
4
5
t [s]

Joint Space Control vs Cartesian Space Control

*** Goal is a Cartesian position**



Linear Control in Joint Space



Linear Control in Cartesian Space

Useful Resources

- VideoLecture by Oussama Khatib: http://academicearth.org/ courses/introduction-to-robotics http://www.virtualprofessors.com/introduction-to-robotics-stanford-cs223a-khatib
(focus on kinematics, dynamics, control)

- OliverBrock'slecturehttp://courses.robotics.tu-berlin.de/ mediawiki/index.php/Robotics:_Schedule_WT09

- Stefan Schaal's lecture Introduction to Robotics: http://www-clmc. usc.edu/Teaching/TeachingIntroductionToRoboticsSyllabus
(focus on control, useful: Basic Linear Control Theory (analytic solution to simple dynamic model → PID), chapter on dynamics)

- Chris Atkeson's "Kinematics, Dynamic Systems, and Control" http: //www.cs.cmu.edu/˜cga/kdc/
(uses Schaal's slides and LaValle's book, useful: slides on 3d kinemat- ics http://www.cs.cmu.edu/˜cga/kdc/ewhitman1.pptx)

- CMU lecture"introduction to robotics"http://www.cs.cmu.edu/afs/ cs.cmu.edu/academic/class/16311/www/current/ syllabus. html
(useful: PID control, simple BUGs algorithms for motion planning, non- holonomic constraints)

- *HandbookofRobotics*(partiallyonlineatGooglebooks)http://tiny. cc/u6tzl

- LaValle's*PlanningAlgorithms*http://planning.cs.uiuc.edu/

Next Time

- System Properties